Rumos



Java Backend com Spring Boot

Novembro de 2026

Siga-nos —

C2 General

Agenda da Formação

01. Spring Boot & Rest Básico

02. API Design, DTOs & Validação

03. Persistência com JPA & Hibernate

04. Integração Externa (TMDB)

05. Qualidade, Erros & Testes

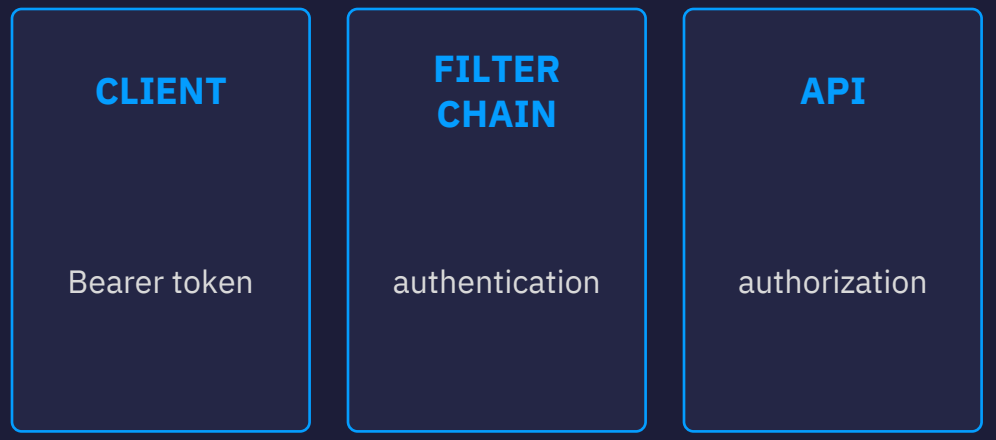
06. Spring Security, JWT & OAuth2

07. Empacotamento & Container

08. Kubernetes Local (Minikube)

Spring Security, JWT e OAuth2

Resultado: MyMovies valida tokens e controla acesso.



OBJETIVO

MyMovies: o acesso que vamos controlar

Operação	Regra da aula	Como verificar
GET /api/movies	Pública	Sem token funciona
POST /api/ratings	Autenticada	Token válido chega ao controller
/api/admin/**	SCOPE_movies.admin	Sem scope: 403

Rumos

A cadeia de segurança da MyMovies

NO ARRANQUE



EM CADA REQUISIÇÃO



Rumos

Fluxo simplificado de um endpoint protegido. Outros filtros e o DispatcherServlet foram omitidos.

SecurityConfig, HttpSecurity e SecurityFilterChain

ELEMENTO	COMO OBJETO SPRING	QUANDO ATUA
SecurityConfig Nossa classe @Configuration	Bean de configuração. Declara métodos @Bean para criar componentes.	No arranque, o Spring processa a classe e chama o método da cadeia.
HttpSecurity Classe do Spring Security	Builder recebido no parâmetro do método. Reúne opções e regras.	Durante a configuração, http.build() constrói a cadeia de filtros.
SecurityFilterChain Interface do Spring Security	O objeto retornado vira um bean. Contém filtros e o critério de seleção.	Em cada requisição, FilterChainProxy seleciona a cadeia e executa filtros.

Bean = objeto registado e gerido pelo Spring. @Bean regista o retorno do método.
O builder monta a cadeia no arranque. Depois, os filtros protegem cada requisição.

CONCEPT

Autenticação e autorização

AUTHENTICATION

Quem faz a requisição?

O token precisa estabelecer uma identidade confiável.

Sem token válido: 401.

AUTHORIZATION

Essa identidade pode realizar a operação?

A regra do endpoint exige permissões específicas.

Sem a authority necessária: 403.

EXEMPLO GUIADO

Mesmo usuário, operações com respostas diferentes

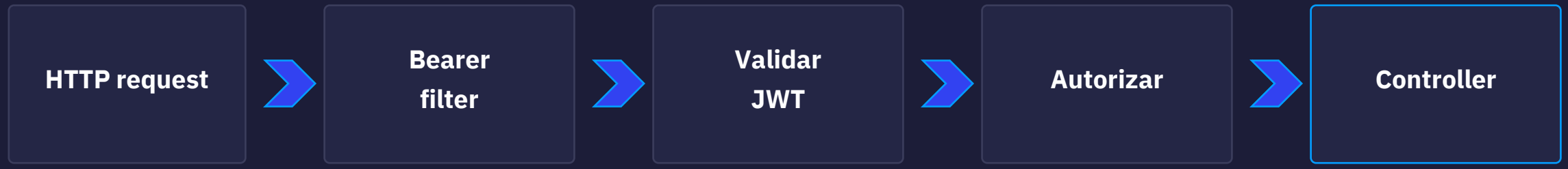
Ana quer consultar a sua watchlist e administrar o catálogo.

Chamada	O que a API sabe	Resultado
Ratings sem token	Ainda não há identidade válida	401
Ratings com token válido	Identidade validada	Acesso autorizado
Admin com token de leitura	Identidade válida, scope insuficiente	403
Admin com movies.admin	Identidade e permissão adequadas	Acesso autorizado

Rumos

DIAGRAM

SecurityFilterChain: segurança antes do controller



Token inválido: 401. Permissão insuficiente: 403.
Nos dois casos, o controller protegido não executa.

@Bean registra a cadeia. HttpSecurity constrói as regras aplicadas às requisições.

Rumos

CONCEPT

Stateless: identidade em cada requisição

STATELESS SECURITY

O cliente envia o token em cada chamada.
A API valida o token e cria o contexto da requisição.
STATELESS evita usar a sessão para guardar autenticação.

Rumos

CONCEPT

JWT e OAuth2: formato e protocolo

JWT

JWT é um formato de token.

Neste exemplo, o access token contém claims e uma assinatura.

OAuth2

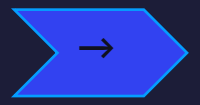
OAuth2 organiza a autorização entre cliente, servidor de autorização e API.

O access token também pode ter outro formato.

Rumos

CONCEPT

OAuth2 por baixo: chave privada assina, chave pública verifica

CHAVE PRIVADA  **CHAVE PÚBLICA**

- O emissor assina o token com a chave privada, que nunca sai dele.
- A API só precisa da chave pública para confirmar assinatura e integridade.
- Assim, OAuth2 funciona igual para qualquer aplicação: a API confia no emissor, não no cliente.
- No projeto MyMovies: classroom-public.pem. Em produção: chaves publicadas pelo emissor.

DIAGRAM

No dia a dia: entrar num site com a conta Google

1 · SITE TERCEIRO

Client: não conhece a sua senha. Redireciona você para o Google com o pedido de acesso (scopes).

2 · GOOGLE

Authorization Server: autentica você, pede consentimento e emite o token assinado com a sua chave privada.

3 · VALIDAÇÃO

O site recebe o token e verifica a assinatura com a chave pública do Google. É o mesmo princípio do MyMovies.

Diferença no lab: a emissão é offline e a chave pública é local; no Google real, o login e as chaves públicas vêm do emissor.

Rumos

DIAGRAM

Os papéis OAuth2 nesta aula

CLIENT

Bruno ou curl envia o access token nas chamadas.

AUTHORIZATION SERVER

No lab, a emissão será feita offline.

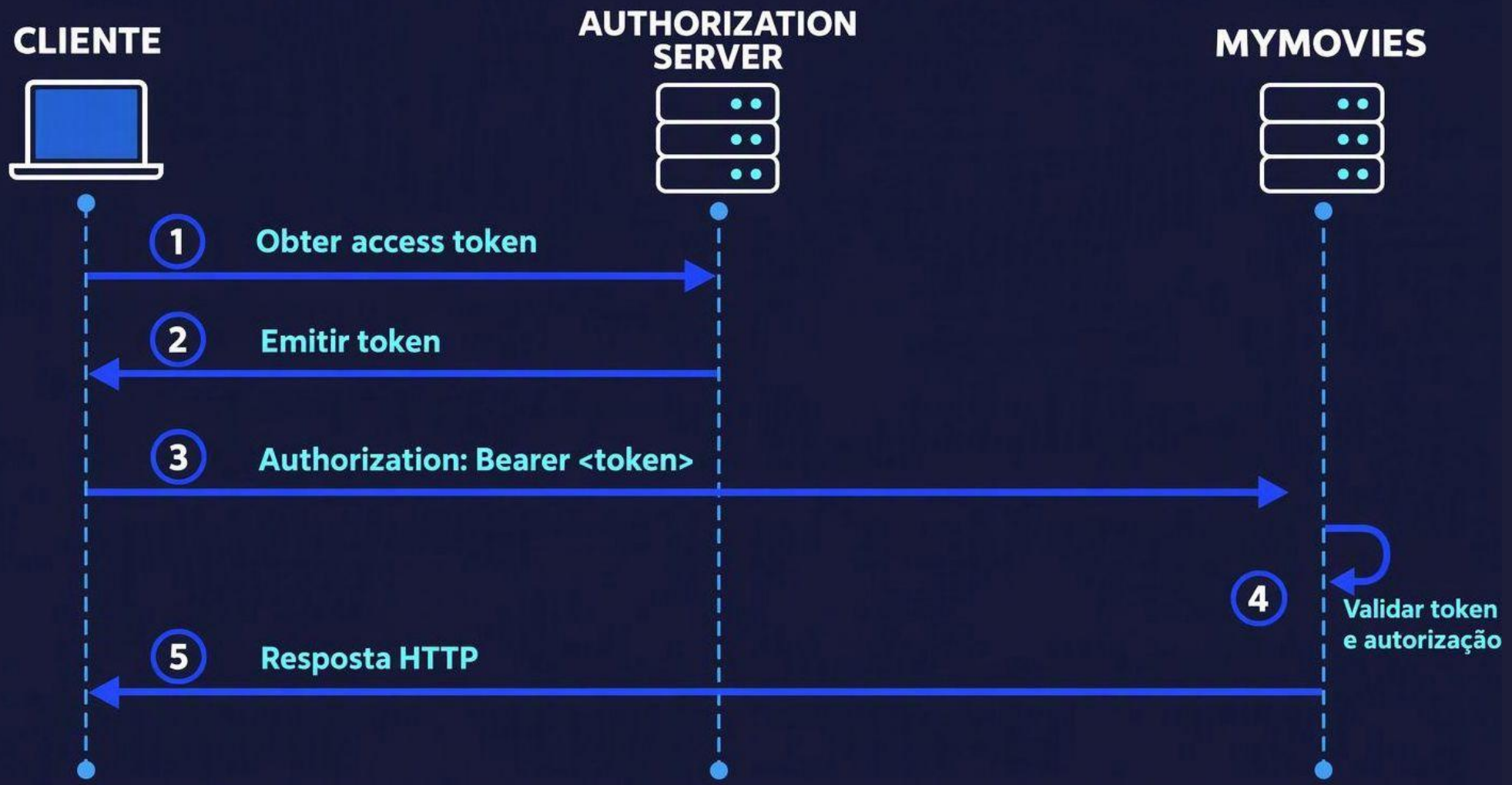
RESOURCE SERVER

MyMovies recebe a chamada, valida o token e aplica as regras.

Emissão simulada, sem login OAuth2. O próximo diagrama mostra o fluxo real.

Rumos

O percurso do access token



MyMovies valida o token.
O login acontece no servidor de autorização.

Rumos

CONCEPT

JWT: estrutura, claims e assinatura

HEADER

alg: algoritmo
kid: identificador da chave

PAYLOAD / CLAIMS

iss: emissor
sub: sujeito
aud: destino
exp: validade
scope: permissões

SIGNATURE

Verifica integridade e origem.
No exemplo, a API usa a chave pública do emissor.

Decodificar permite ler. Validar estabelece confiança.

Rumos

Claims: uma identidade e um contrato de uso

PAYLOAD SINTÉTICO, SEM TOKEN REAL

```
{
  "iss": "https://issuer.example",
  "sub": "aluno-42",
  "aud": "mymovies",
  "exp": 2000000000,
  "scope": "movies.read"
}
```

iss origem que a API aceita

sub identificador do sujeito

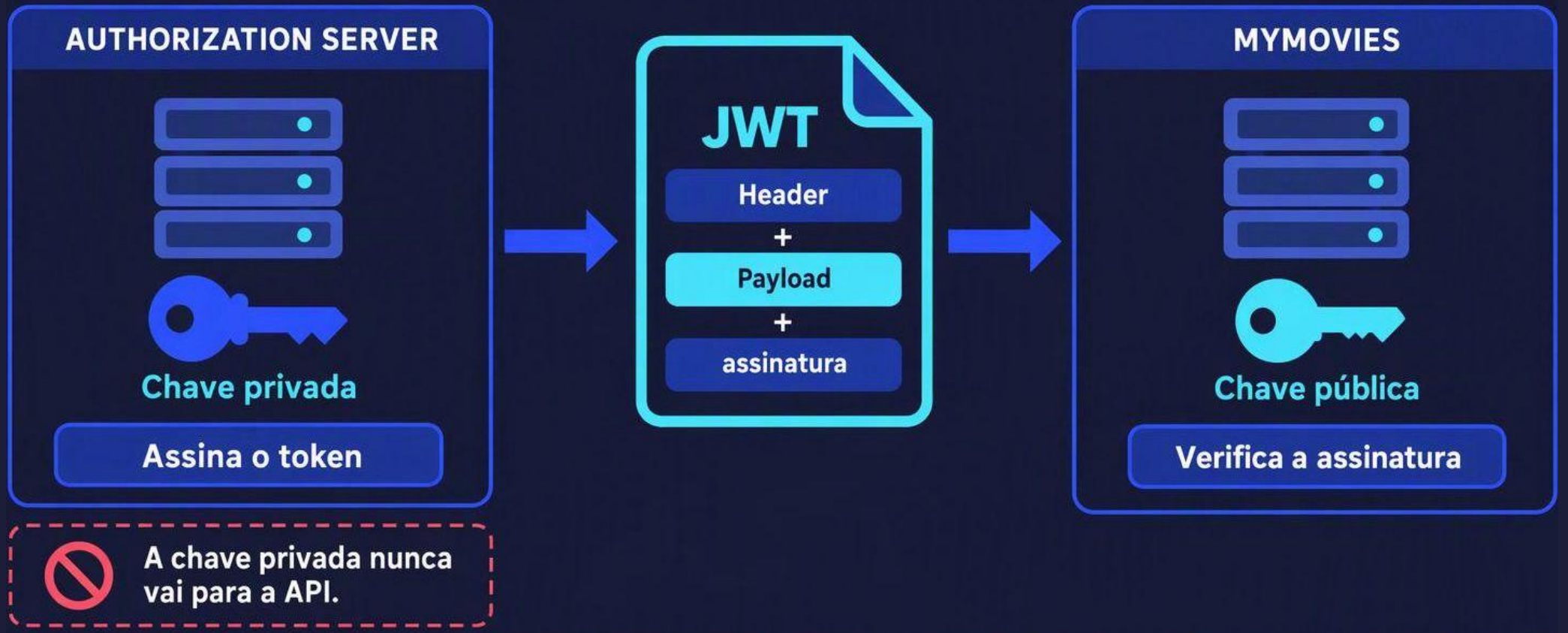
aud API destinatária

exp limite temporal, em segundos Unix

scope capacidades concedidas

Ler o JSON não valida a assinatura. Nunca coloque senhas ou dados sensíveis no payload.

Assinatura: prova de integridade e origem



Assinatura não esconde o payload.

Rumos

CONCEPT

Issuer e chave pública no laboratório local

`iss conhecido`  **urn:mymovies:classroom**

- O formador assina tokens com a chave privada de laboratório.
- A API recebe apenas classroom-public.pem.
- O decoder valida RS256, iss, aud e validade.
- O issuer é um identificador fixo, sem descoberta remota.

Rumos

Validação: as perguntas que o decoder responde

Verificação	Pergunta	Quando rejeitar
Assinatura + algoritmo	As claims mantêm integridade?	Chave ou algoritmo não aceitos
iss	O emissor é o configurado?	Origem diferente
exp / nbf	O token está na janela válida?	Expirado ou ainda não válido
aud	Foi emitido para esta API?	Destino diferente, quando validado

Audience precisa de uma política explícita. O token válido para outra API não deve bastar.

Segurança de APIs e web: onde cada defesa atua

Camada	Pergunta	Defesa principal
Navegador	Que frontend pode ler a API? Como evitar ações e scripts indevidos?	CORS, proteção CSRF e prevenção de XSS
Identidade	Quem chama e com que permissões?	Validar JWT e exigir authorities
Dados	Pode aceder a este objeto? Os dados alteram uma consulta?	Autorização por objeto e parâmetros seguros
Operação	Como reduzir exposição e abuso?	HTTPS, limites de uso e logs sem segredos

CORS (Cross-Origin Resource Sharing) : que frontend pode ler a resposta da API?

ORIGENS DIFERENTES

Frontend: localhost:5173

API: localhost:8080

A porta muda a origem.

O navegador exige permissão CORS para o JavaScript ler a resposta.

PERMISSÃO EXPLÍCITA

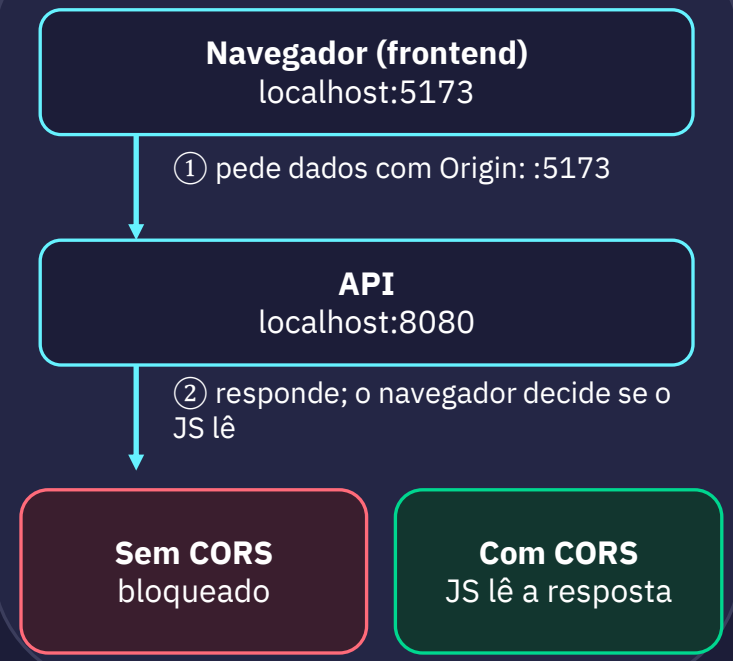
Permitir só as origens necessárias.

Limitar métodos e headers.

OPTIONS verifica a permissão antes do pedido real.

Se cookies/credenciais forem permitidos, não use allowedOrigins wildcard (*)
Defina origem exata.

COMO O NAVEGADOR DECIDE



Bruno/curl não aplicam CORS. A API continua a exigir autenticação e autorização.

Rumos

CSRF: uma ação que o utilizador não pediu

CSRF = Cross-Site Request Forgery

Em português: falsificação de pedido entre sites

Exemplo

1. Ana faz login numa MyMovies que usa cookie de autenticação.
2. Outra página induz o navegador a enviar uma ação à MyMovies.
3. O navegador pode anexar o cookie de login automaticamente.
4. Sem proteção, a API pode executar a ação em nome de Ana.



Proteção

A aplicação fornece um valor imprevisível: o token CSRF.

O frontend envia esse valor nas ações e a API valida-o.

Se faltar ou estiver errado, a API rejeita o pedido.

O cookie identifica a sessão. O token CSRF ajuda a impedir pedidos forjados.

Rumos

XSS: um comentário que executa código no navegador

XSS = Cross-Site Scripting

Em termos simples: injeção de script numa página web

Exemplo

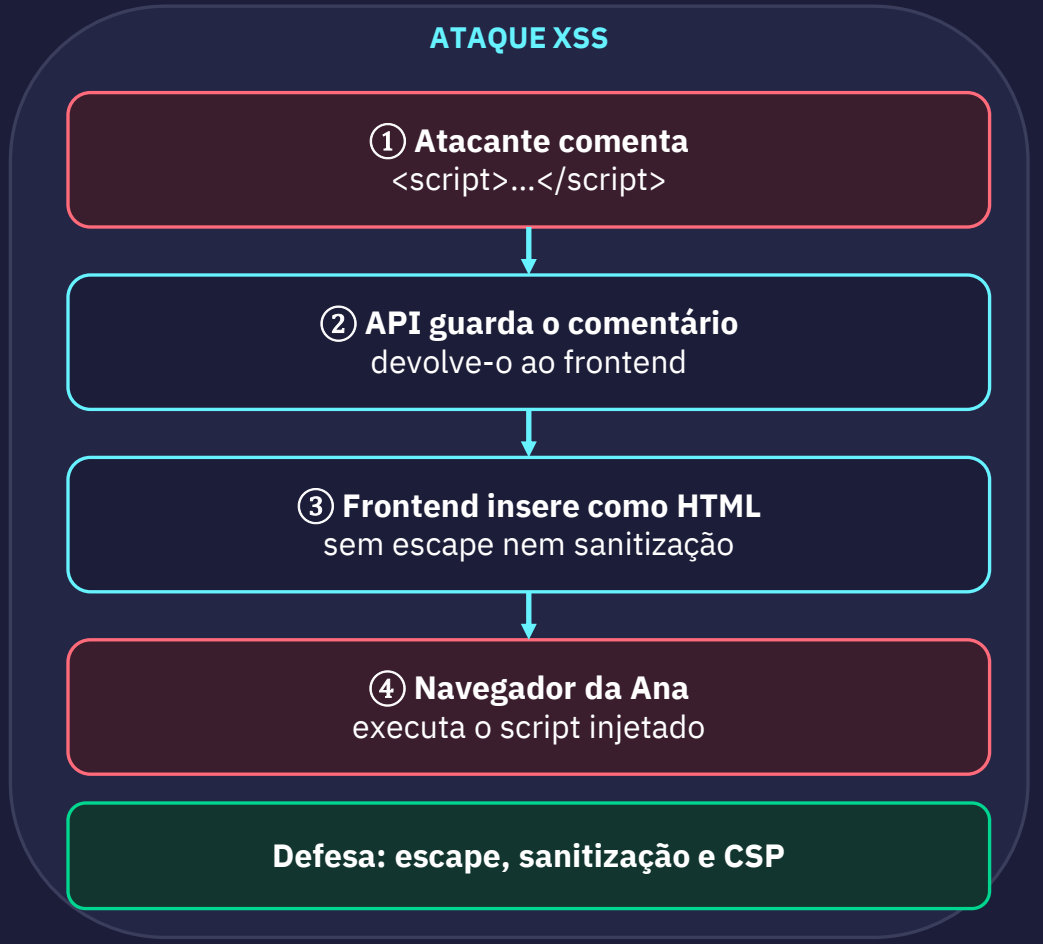
1 Numa MyMovies com comentários, alguém publica conteúdo com código.

2 A API guarda o comentário e devolve-o ao frontend.

3 O frontend insere esse conteúdo na página como HTML sem proteção.

4 Ana abre a página e o código executa no navegador dela.

Pode alterar a página ou realizar ações em nome de Ana.



Proteção

Renderizar comentários como texto, com escape/encoding correto.

Se HTML for necessário, sanitizar antes de o mostrar.

CSP (Content Security Policy) acrescenta uma camada de proteção à execução de scripts.

HttpOnly reduz a leitura de cookies por scripts. XSS ainda pode executar ações.

Rumos

Proteção dos dados: quatro erros comuns e como evitá-los

Risco	O que pode acontecer	Como o backend se defende
Acesso a objetos	Um utilizador consegue ver a info de outra pessoa.	Verificar sempre quem é o dono do dado.
Injeção SQL	O texto escrito pelo utilizador vira parte do comando SQL.	Usar parâmetros, nunca juntar texto à query.
Campos indevidos	O pedido tenta definir o dono ou ganhar permissões de admin.	Aceitar só os campos permitidos. A identidade vem do JWT.
Dados inválidos	Chega uma avaliação fora do intervalo permitido.	Validar sempre no servidor. O frontend não basta.

Do token à regra: scope vira authority

```
1. No token (JWT)
scope = "movies.read movies.admin"

2. O Spring converte (prefixo SCOPE_)
SCOPE_movies.read
SCOPE_movies.admin

3. A regra no filtro compara
.hasAuthority("SCOPE_movies.admin")
```

Scope = o crachá



Authority = o que o porteiro lê

hasAuthority = o porteiro: só deixa passar se o texto for exatamente igual.

Regra pede SCOPE_movies.admin e o token só tem movies.read? Resultado: 403. Roles exigem configuração própria.

CODE · SECURITYCONFIG.JAVA

Configuração Central: Descodificando as Anotações de Segurança

```
@Configuration
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {

  // @Bean SecurityFilterChain
  // @Bean JwtDecoder
}
```

@Configuration

Define a classe como fonte de definições de Beans para o container IoC do Spring.

@EnableWebSecurity · segurança perimetral (HTTP)

Ativa o suporte de segurança Web e a cadeia de filtros HTTP (SecurityFilterChain). Regras por URIs e métodos HTTP (requestMatchers).

@EnableMethodSecurity · segurança granular (domínio)

Ativa @PreAuthorize e @PostAuthorize nos métodos. Valida permissões na camada de serviço (@Service) ou nos métodos do controller.

Perimetral: quem pode chamar cada URI. Granular: quem pode executar cada método.

Rumos

CODE

SecurityFilterChain no Spring Security 7

```
@Bean
SecurityFilterChain security(HttpSecurity http) throws Exception {
  return http
    .csrf(csrf -> csrf.disable())
    .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
    .authorizeHttpRequests(a -> a
      .requestMatchers(GET, "/api/movies", "/api/movies/**",
          "/actuator/health").permitAll()
      .requestMatchers("/api/admin/**")
        .hasAuthority("SCOPE_movies.admin")
      .anyRequest().authenticated())
    .oauth2ResourceServer(o -> o.jwt(withDefaults()))
    .build();
}
```

- 1 GET de filmes e health: públicos

2 Admin: exige a authority SCOPE_movies.admin (vem do scope do token)

3 Restantes: autenticados

4 JWT: validação do Resource Server

CSRF desligado apenas neste lab: Bearer no header, sem login por cookies.

Rumos

Anatomia do filtro (1/2): CSRF, sessão e regras

Elemento	O que faz	Neste lab
csrf(...)	Proteção contra pedidos forjados que usam cookies	Desligado: o token vai no header, não em cookie
sessionManagement(...)	Define se o servidor cria sessão HTTP	STATELESS: sem sessão; cada pedido traz o JWT
authorizeHttpRequests(...)	Lista de regras de acesso por pedido	Aqui vive toda a política de autorização
requestMatchers(...)	Escolhe os pedidos da regra: método, caminho, **	Avaliadas por ordem: a primeira que corresponde ganha

Anatomia do filtro (2/2): acesso, autenticação e JWT

Elemento	O que faz	Neste lab
@Bean SecurityFilterChain	Regista a cadeia de filtros que intercepta cada pedido; build() constrói-a	Uma cadeia por política de segurança
permitAll() / authenticated()	Liberta o acesso ou exige identidade válida	GET de filmes e health: públicos; restantes: autenticados
hasAuthority("SCOPE_movies.admin")	Exige uma permissão concreta; SCOPE_ vem dos scopes do JWT	/api/admin/** só com movies.admin
oauth2ResourceServer(...jwt...)	A API valida o Bearer token: assinatura, emissor e validade	withDefaults() usa a configuração da aplicação

Outras peças comuns numa SecurityFilterChain

Elemento	Para que serve	Exemplo
cors(...)	Permite que um frontend de outra origem chame a API	withDefaults() + CorsConfigurationSource
exceptionHandling(...)	Personaliza as respostas 401 e 403	authenticationEntryPoint, accessDeniedHandler
headers(...)	Cabeçalhos de segurança nas respostas	HSTS, Content-Security-Policy
securityMatcher / addFilterBefore	Limita a cadeia a caminhos; insere filtros próprios	Várias cadeias com @Order; filtro de API key

CONCEPT

401 e 403: problema diferente, resposta diferente

401 UNAUTHORIZED

Token ausente, expirado, assinatura inválida ou issuer incorreto.

A API não aceita a credencial.

Investigue o token e WWW-Authenticate.

403 FORBIDDEN

Token válido, mas falta uma permissão exigida.

No lab: falta movies.admin.

Um 403 também pode vir de CSRF. Use evidência para distinguir.

Rumos

Matriz de testes: previsões que viram evidências

Requisição	Credencial	Esperado
GET movies / health	Sem token	2xx do endpoint existente
POST ratings	Válido + body válido	Resposta prevista pelo endpoint
GET admin/stats	Válido, só movies.read	403
GET admin/stats	Válido, movies.admin	Sem 401/403

LAB

Lab Core: proteger e explicar a rejeição

Core

1. Preserve GET /api/movies público.
2. Proteja POST /api/ratings.
3. Execute sem token e com token válido.
4. Explique o 401 e confirme o controller.

Rumos

CHECKPOINT

Definition of Done

- [x] GET movies + health + metrics: públicos
- [x] Ratings: autenticado
- [x] Assinatura, exp, iss ou aud inválidos: 401
- [x] Admin sem authority: 403
- [x] Admin com authority: permitido

Também verificar: nenhuma credencial ou token no Git.

Rumos

DEEP DIVE

Access token: JWT vs opaque e outros formatos

JWT (self-contained)

Contém claims e assinatura.

A API valida localmente. Revogação difícil antes de expirar.

Opaque (by-reference)

String aleatória, sem significado.

A API valida por introspection. Revogação imediata.

Outros formatos

JWE: JWT cifrado, o cliente não lê os claims. PASETO: alternativa ao JWT com menos opções de algoritmo.

Macaroons e SAML assertions: usos específicos. mTLS / DPoP: ligam o token ao cliente, sobre opaque ou JWT.

Rumos

Deep dive: antídotos operacionais e evidências para diagnóstico

Medida	O que reduz	Evidência a procurar
HTTPS / TLS	Interceção de tokens e alteração do tráfego.	HTTPS fora do localhost didático. Certificados e configuração revistos.
Limites de uso	Abuso de pedidos e consumo excessivo de recursos.	Paginação, tamanho/timeout e rate limit. Quando aplicado: 429.
Tokens e segredos	Exposição e uso indevido de credenciais.	Prazo de validade, chaves protegidas e ausência de tokens em Git/logs.
Diagnóstico web	Confusão entre CORS, CSRF e autorização.	DevTools: OPTIONS e headers. API: status, regra e log sanitizado.
Manutenção	Falhas conhecidas e exposição desnecessária.	Dependências atualizadas e endpoints/configuração revistos.